

RELATÓRIO TÉCNICO – UNIDADE 2

Marcos Paulo S. B. Silva

Disciplina: Introdução às Técnicas de Programação (ITP)

2025

1 Introdução

Este relatório apresenta o desenvolvimento e a evolução do projeto "Gerenciador de Tarefas" ou To-Do List, desenvolvido na linguagem C, referente à Unidade 2 da disciplina. O projeto consiste em uma aplicação de linha de comando (CLI) que permite ao usuário gerenciar suas atividades diárias.

Os objetivos principais desta unidade foram a refatoração do código para utilização de alocação dinâmica de memória, a implementação de estruturas de dados mais complexas (como matrizes para relatórios) e o uso aprofundado de ponteiros e manipulação de strings.

Dentre as novas funcionalidades implementadas, destacam-se: a busca de tarefas por palavra-chave, a remoção de tarefas com liberação de memória, e a geração de um relatório visual de produtividade utilizando matrizes e laços aninhados.

2 Metodologia

Para o desenvolvimento deste projeto assim como na U1, utilizou-se a linguagem C, compilada através do GCC. O ambiente de desenvolvimento foi o VS Code, configurado para compilação de múltiplos arquivos.

A abordagem de desenvolvimento seguiu o paradigma de programação modular. O código foi organizado em três arquivos principais para garantir a manutenibilidade e a separação de responsabilidades:

- **main.c:** Ponto de entrada da aplicação, responsável pela alocação inicial, o loop principal do menu e a orquestração das chamadas de função.
- **tarefas.c:** Contém a implementação lógica das funcionalidades (adicionar, listar, deletar, buscar e relatar).
- **tarefas.h:** Arquivo de cabeçalho que expõe os protótipos das funções e definições de constantes, permitindo a comunicação entre o *main* e as implementações.

3 Análise do Código

Nesta seção, detalha-se como os conceitos chave da Unidade 2 foram aplicados no projeto.

3.1 Strings

As strings foram fundamentais para armazenar as descrições das tarefas. Diferente de tipos primitivos simples, strings em C são vetores de caracteres terminados em nulo (`'\0'`).

- **Funções Utilizadas:** Implementou-se a função `fgets` para leitura segura da entrada do usuário, evitando estouro de buffer. A função `strcspn` foi usada para remover o caractere de nova linha (`'\n'`) capturado pelo `fgets`. Para copiar a string do buffer temporário para a memória alocada dinamicamente, utilizou-se `strcpy`. Na funcionalidade de busca, a função `strstr` foi essencial para verificar se uma substring (termo de busca) estava contida na descrição da tarefa.

3.2 Estruturas de Repetição Aninhadas

As estruturas de repetição aninhadas (loops dentro de loops) foram aplicadas na funcionalidade de relatório (`exibirRelatorio`).

- **Aplicação:** Um loop externo percorre os 12 meses do ano, enquanto um loop interno percorre os 31 dias de cada mês.
- **Complexidade:** A complexidade do algoritmo é $O(M \times D)$, onde M é o número de meses e D é o número de dias, garantindo que toda a matriz de relatório seja visitada para exibir a contagem de tarefas por dia.

3.3 Matrizes

Para o controle estatístico das tarefas, utilizou-se uma matriz bidimensional de inteiros: `int relatorioTarefas[12][31]`.

- **Implementação:** As linhas representam os meses (0 a 11) e as colunas representam os dias (0 a 30).
- **Operações:** A matriz é inicializada com zeros no início do programa. Quando uma tarefa é adicionada, a célula correspondente é incrementada: `relatorioTarefas[mes-1][dia-1]++`. Isso permite um acesso direto e eficiente $O(1)$ para atualizar e consultar dados estatísticos.

3.4 Ponteiros

Os ponteiros foram o pilar central desta etapa do projeto, utilizados para manipulação direta de memória e passagem de parâmetros por referência.

- **Uso Prático:** A lista de tarefas foi definida como `char **tarefas`, um ponteiro para ponteiro, permitindo criar uma lista de strings.
- **Passagem por Referência:** O contador de tarefas (`int *contador`) foi passado por referência para as funções `adcTarefa` e `deletarTarefa`. Isso permitiu que essas funções modificassem o valor da variável original declarada na `main`, garantindo que o estado do programa fosse mantido corretamente entre diferentes chamadas.

3.5 Alocação Dinâmica

A alocação dinâmica foi necessária para otimizar o uso de memória, permitindo que o programa usasse apenas o necessário para armazenar as tarefas.

- **Gerenciamento:** Utilizou-se `malloc` para alocar o array principal de ponteiros e, posteriormente, para alocar a memória exata de cada string de descrição de tarefa.
- **Prevenção de Vazamentos:** Para evitar *memory leaks*, implementou-se a função `free`. Na remoção de uma tarefa, a memória específica daquela string é liberada imediatamente. Ao encerrar o programa, um laço percorre todas as tarefas restantes liberando-as individualmente antes de liberar o array principal. Falhas de alocação são tratadas verificando se o retorno do `malloc` é `NULL`.

4 Dificuldades e Soluções

Durante o desenvolvimento, o principal desafio técnico foi o gerenciamento correto dos ponteiros e a integração entre múltiplos arquivos.

Inicialmente, houve dificuldades com erros de *"undefined reference"*, causados pela tentativa de compilar apenas o arquivo `main.c` sem vincular o `tarefas.c`. A solução envolveu o entendimento do processo de *linking* do GCC e a configuração correta do comando de compilação (`gcc src/main.c src/tarefas.c`).

Outro desafio foi a manipulação de ponteiros para ponteiros (`char **`). A lógica para deletar uma tarefa e reorganizar o array (shift dos elementos) exigiu cuidado para evitar acesso a posições inválidas de memória (*segmentation fault*) e garantir que não houvesse ponteiros pendentes (*dangling pointers*), resolvido ao atribuir `NULL` à última posição após a reorganização.

5 Conclusão

A Unidade 2 representou um avanço significativo em relação à versão estática do projeto. A transição para alocação dinâmica trouxe eficiência, pois o programa não reserva mais memória excessiva para descrições curtas ou tarefas inexistentes.

A organização modular do código facilitou a manutenção e a depuração, isolando a lógica de negócios da interface do usuário. O uso de matrizes permitiu agregar dados de forma estruturada, enriquecendo as funcionalidades do sistema com relatórios visuais.

Se tivesse a continuação do projeto na próxima unidade, o foco seria a persistência de dados. Atualmente, as tarefas são perdidas ao fechar o programa. Essa implementação de manipulação de arquivos permitiria salvar o estado da lista e do relatório, completando o ciclo de vida de um gerenciador de tarefas funcional e robusto.